

SC102 L02: C# 进阶（下）——详细学习笔记

涵盖：泛型 | 委托与事件 | Action 与 Func | 字段与属性 | 接口 | 部分类 | Lambda 表达式 | 隐式类型

1 泛型（Generics）

1.1 什么是泛型？

[p.4]

泛型的标志性特征就是尖括号 < >。在 Unity 中我们已经频繁使用泛型：

```
GetComponent<SpriteRenderer>();  
Resources.Load<AudioClip>();
```

通俗理解：泛型允许一个函数或类接受类型作为参数。就像普通参数让函数接受不同的数据值，泛型参数让函数/类接受不同的数据类型。

为什么需要泛型？考虑一个将两个数相加的函数：

```
int Add(int a, int b) { return a + b; }
```

这个函数只能接受 `int` 类型。如果要支持 `float`、`double`、`long`，难道要每种类型写一个重载吗？泛型解决了这个问题：编写一次代码，适配多种类型。

1.2 泛型方法（Generic Methods）

[p.5]

定义泛型方法时，在函数名后用尖括号声明类型参数：

```
// 非泛型  
int Add(int a, int b) {  
    return a + b;  
}
```

```
// 泛型
T AddGeneric<T>(T a, T b) {
    return a + b;
}
```

这里 `T` 是一个类型变量 (**Type Parameter**)。T 是惯例命名 (Type 的首字母)，也可以用任何有效标识符。

在调用泛型方法时，需要指定类型参数：

```
float sum = AddGeneric<float>(5f, 7f); // 完整写法
float sum = AddGeneric(5f, 7f);       // 简略写法 (编译器自动推断)
```

编译器可以从传入的实参 (5f 和 7f 都是 float) 推断出 `T = float`，因此第二种写法也是合法的。

1.3 多类型参数的泛型方法

[p.6]

泛型方法可以接受多个类型变量，用逗号分隔：

```
void Insert<TKey, TValue>(Dictionary<TKey, TValue> dict, TKey key, TValue value) {
    dict.Add(key, value);
}
```

```
// 调用
Insert<int, string>(dict, 2, "February"); // 完整
Insert(dict, 2, "February");              // 编译器推断
```

从这个例子可以推断 Unity 的 `GetComponent` 方法的签名大概是：

```
T GetComponent<T>() where T : Component {
    T comp;
    // ... 查找并返回对应Component
    return comp;
}
```

思考：我们可以 `GetComponent<int>()` 吗？不可以！因为 `int` 不是 `Component` 的子类。这就是接下来要讲的类型限制的作用。

1.4 泛型的类型限制 (Type Constraints)

[p.7]

有些时候，类型参数不能是任意类型。比如 `AddGeneric<T>` 中的 `+` 运算符并不是在所有类型间都定义了 (`bool + bool` 没有意义)。再比如 `GetComponent<T>` 只能接受 `Component` 及其子类作为 `T`。

使用 `where` 关键字对类型变量做出限制：

```
// GetComponent: T必须是Component的子类
T GetComponent<T>() where T : Component {
    T comp;
    // ...
    return comp;
}
```

常见的类型限制 (约束)：

- `where T : struct` —— `T` 必须是值类型 (如 `int`, `float`, `Vector3`)
- `where T : class` —— `T` 必须是引用类型 (如 `GameObject`, `MonoBehaviour`)
- `where T : new()` —— `T` 必须有一个无参数的构造函数
- `where T : < 基类名 >` —— `T` 必须继承自指定基类
- `where T : < 接口名 >` —— `T` 必须实现了指定接口
- 多个约束可以组合: `where T : class, new()` 表示 `T` 必须是引用类型且有无参构造函数

练习：为 `AddGeneric` 添加正确的类型约束：

```
// 数值类型才支持+运算符
T AddGeneric<T>(T a, T b) where T : struct {
    return a + b; // 仍然有问题 (并非所有struct都支持+),
                // 实际中通常用 where T : INumber<T> (.NET 7+)
}
```

1.5 泛型类 (Generic Classes)

[p.8]

类也可以有类型参数，思想和泛型方法一致：

```
public class MyList<T> where T : struct {
    List<T> list = new List<T>();

    public void Add(T item) {
        list.Add(item);
    }
}
```

```

    }
}

// 使用
MyList<int> intList = new MyList<int>();
intList.Add(5);
MyList<float> floatList = new MyList<float>();
floatList.Add(3.14f);

```

泛型类在编写通用数据结构时非常常用——`List<T>`、`Stack<T>`、`Queue<T>`、`Dictionary<TKey, TValue>` 全都是泛型类。

2 委托 (Delegate)

2.1 什么是委托？

[p.9]

在 SC-101 中我们认识了 `Action` 和 `Func`——“可以存放函数的容器”。它们背后的机制就是委托 (Delegate)。委托的本质是：一种可以存放函数的类型，和 C 语言中的函数指针非常相似。

定义委托类型的语法：

```
delegate <返回值类型> <委托名>(<参数列表>);
```

具体示例：

```

// 定义一个委托类型：接受两个int参数，返回int
delegate int Calculate(int x, int y);

// 使用委托
Calculate cal;                // 声明委托变量
cal = AddInt;                 // 将函数"装进"变量
int sum = cal(6, 7);          // 通过委托调用函数

int AddInt(int x, int y) {
    return x + y;
}

```

关键理解：上面定义的是一个类型（类似 `class`），使用时需要声明该类型的变量。`cal` 就是 `Calculate` 类型的一个变量，其中存放了 `AddInt` 函数的引用。

2.2 委托的 Multicasting (多播)

[p.10]

一个委托变量可以同时存储多个函数，这叫做**多播 (Multicasting)**。

```
Calculate cal;
cal += AddInt;           // 添加函数
cal += MinusInt;         // 再添加一个函数
cal -= AddInt;           // 移除函数
```

多播委托的调用规则：

- 当委托被调用时，其中的函数按添加顺序**依次执行**
- 如果参数中有引用类型，该变量会依次受到这些函数的影响（前一个函数的结果传给下一个）
- 如果函数有返回值，最终返回的是**最后一个函数**的返回值

调用列表 (Invocation List)：委托中存储的函数列表可以通过 `GetInvocationList()` 获取：

```
cal.GetInvocationList();           // 获取所有已注册的函数
cal.GetInvocationList().Length;    // 获取注册函数的数量
```

2.3 练习：多播委托输出分析

[p.11]

```
delegate int Calculate(int x, int y);

public static void Main() {
    Calculate cal = null;
    cal += Add;
    cal += Add;
    int sum = cal(1, 1);
    Console.WriteLine(sum);
}

private static int Add(int x, int y) {
    return x + y;
}
```

答案：2

分析： Add 被添加了两次到委托中。调用 `cal(1, 1)` 时：

1. 第一次调用 `Add(1, 1)`，返回 2（但被丢弃，因为不是最后一个）
2. 第二次调用 `Add(1, 1)`，返回 2（这是最后一个函数的返回值，赋值给 `sum`）

因此 `sum = 2`。注意：两个函数都被执行了，但只有最后一个的返回值被保留。

2.4 事件 (Event)

[p.12]

事件 (Event) 是对委托的一层封装。它使得我们可以在不同类之间方便地“发送提醒”。

简而言之，事件就是一个只支持 `+=` 和 `-=` 的委托，禁止了直接赋值 (`=`)，从而防止外部代码意外地覆盖整个 invocation list。

```
public delegate void MyDelegate(string message);
public event MyDelegate MyEvent;    // 用event关键字声明事件

// 使用
MyEvent += SomeHandler;    // 可以
MyEvent -= SomeHandler;    // 可以
MyEvent = SomeHandler;     // 不可以！编译错误！
```

事件的优势：

- 封装性：外部类只能订阅/取消订阅，不能直接操控或清空事件
- 安全性：防止一个订阅者误覆盖其他订阅者的注册
- 清晰的意图：事件明确传达“这是一个通知机制”的设计意图

2.5 Action 和 Func 的真面目

[p.13]

了解了委托，`Action` 和 `Func` 就变得非常简单——它们只是预定义的泛型委托类型！

```
// Action = 无返回值的委托（0到16个参数）
public delegate void Action();
public delegate void Action<in T>(T obj);
public delegate void Action<in T1, in T2>(T1 arg1, T2 arg2);
// ... 直到16个参数

// Func = 有返回值的委托（0到16个参数 + 1个返回值类型）
public delegate TResult Func<out TResult>();
public delegate TResult Func<in T, out TResult>(T arg);
```

```
public delegate TResult Func<in T1, in T2, out TResult>(T1 arg1, T2 arg2);
// ... 直到16个参数
```

in / out 关键字：

- **in:** 表示该参数只会被函数读取，不会被修改（协变支持）
- **out:** 表示该参数会被函数写入/传出（逆变支持）。在 `Func` 中，`out TResult` 标记了返回值类型的协变特性

在 Unity 中使用：

- 简单回调：`Action callback = () => { Debug.Log("Done!"); };`
- 带参数回调：`Action<int, string>` 接受 `int` 和 `string` 参数
- 带返回值：`Func<float, float> square = x => x * x;`

3 字段与属性 (Field & Property)

3.1 字段 (Field)

[p.15]

字段就是类中的成员变量：

```
public class Player {
    private int speed = 5;    // 这是一个字段
    public string name;       // 这也是一个字段
}
```

3.2 属性 (Property)：字段的封装

[p.15]

很多时候我们不希望其他类直接访问我们的字段——可能是出于安全考虑，也可能希望每次访问字段时执行一些额外逻辑。**属性 (Property)** 提供了这个能力。

属性由 **getter** 和 **setter** 两个访问器 (accessor) 组成：

```
private int speed = 5;

public int Speed {
    get { return speed; }    // getter: 读取时的行为
    set { speed = value; }   // setter: 写入时的行为, value是隐式参数
}
```

一个最标准的属性实现实际上”什么都没做”——`getter` 直接返回字段值，`setter` 直接赋值。但它为后续添加逻辑（验证、计算、日志等）预留了空间。

3.3 Getter 详解

[p.16]

Getter 定义了读取属性的行为，必须有返回值。语法：`get { < 函数语句 > }`

```
private int speed = 5;
```

```
public int Speed {  
    get { return speed; } // 每当你读Speed时，实际上在调用这个getter  
}
```

// 也可以自定义任意行为

```
public int Speed {  
    get { return 999; } // 虽然没意义，但说明了getter的灵活性  
}
```

Getter 的存在允许我们：

- 隐藏内部存储格式（如内部用米，外部显示千米）
- 计算派生值（如面积 = 长 * 宽，每次读取时实时计算）
- 延迟初始化（第一次访问时才创建对象）
- 添加日志或断点，便于调试

3.4 Setter 详解

[p.17]

Setter 定义了写入属性的行为。它有一个隐含的参数 `value`，代表被写入的值。语法：`set { < 函数语句 > }`

微软官方示例解析：

```
private double _seconds;
```

```
public double Hours {  
    get { return _seconds / 3600; } // 读取时：秒转小时  
    set {  
        if (value < 0 || value > 24) // 验证：小时值必须在0-24之间  
            throw new ArgumentOutOfRangeException(  
                nameof(value),  
                "The valid range is between 0 and 24.");  
        _seconds = value * 3600; // 存储时：小时转秒  
    }  
}
```



```
    }  
}
```

分析：这个属性内部使用 `_seconds`（秒）存储时间，但对外提供 `Hours`（小时）的接口。Getter 将秒转换为小时返回；Setter 先验证输入在 0-24 范围内，再将小时转换为秒存储。这就是“内部表示与外部接口分离”的典型例子。

3.5 自动属性 (Auto-Implemented Property)

[p.18]

为每个字段手写完整的 getter/setter 确实繁琐。C# 提供了**自动属性**的简写：

```
public int Speed { get; set; }
```

这行代码等价于：编译器自动生成一个**看不见的私有字段**，并提供默认的 getter 和 setter（就是直接读写的“最标准实现”）。

通过自动属性可以灵活控制可读/写性：

```
public int Speed { get; } // 只读（只能在构造函数中赋值）  
public int Speed { get; private set; } // 可读，但只有本类可以写  
public int Speed { get; protected set; } // 可读，本类和子类可以写
```

注意：属性不可以设置为“可写但不可读”（没有 `set; only` 这样的语法）。

3.6 字段 vs 属性：何时使用哪个？

- 优先使用自动属性（`{ get; set; }`）作为公开接口，因为它为将来添加逻辑预留了空间而不改变调用者代码
- 使用私有字段存储不需要外部访问的内部状态
- 只用字段而不用属性的场景：`ref / out` 参数传递（属性不能作为 `ref/out` 参数）、需要高性能且确定永不需封装逻辑时
- 属性优于公开字段：属性是方法在语法层面的封装，改变内部实现不影响调用者

4 接口 (Interface)

4.1 什么是接口？

[p.19]

接口 (Interface) 定义了一系列类的规范/契约 (**contract**)。接口中声明方法、属性、事件等成员，但不提供实现。任何继承接口的类**必须**提供这些成员的实现。

动机：假设我们在游戏中想添加”可被攻击”的一系列物体（敌人、可破坏的箱子、栅栏等）。它们有共同且必须拥有的特性：受伤方法、生命值等。我们可以建立一个接口将它们统一起来：

```
public interface IDamageable {  
    float hp { get; set; }           // 属性：生命值（只声明，不实现）  
    void OnDamage(float value);      // 方法：受伤（只声明，不实现）  
}
```

接口就像一个”未完成的类”——只定义”做什么”，不定义”怎么做”。

4.2 接口的实现

[p.20]

一个类可以实现（继承）接口，并必须提供接口中所有成员的实现：

```
public interface IDamageable {  
    float hp { get; set; }  
    void OnDamage(float value);  
}  
  
public class Enemy : IDamageable {  
    public float hp { get; set; }      // 实现属性  
  
    public void OnDamage(float value) { // 实现方法  
        hp -= value;  
    }  
}
```

接口与类的关键区别：

- 类只能继承自一个基类（单继承），但可以实现多个接口
- 接口中不能有字段（只有属性和方法声明）
- 接口成员不能有访问修饰符（默认为 public）
- 接口不能有构造函数
- 接口之间也可以相互继承

多接口实现示例：

```
public class Enemy : IDamageable, IHittable, IMovable {  
    // 必须实现所有三个接口的所有成员  
}
```

何时使用接口？当需要一组行为相似但继承体系不同的类具有共同功能时。例如：

- IDamageable ——敌人、箱子、门都可以被打
- IInteractable ——NPC、物品、开关都可以被交互
- ISaveable ——任何需要保存/加载状态的物体

5 部分类（Partial Class）

5.1 概念

[p.21]

部分类（Partial Class）允许我们把同一个类写在多个文件中，每个文件提供类的一部分定义。使用 `partial` 关键字：

```
// 文件1: Map.cs
public partial class Map {
    private float hp { get; set; }
}

// 文件2: MapAddHP.cs
public partial class Map {
    public void AddHP(float value) {
        hp += value;
    }
}
```

这两个文件合在一起的效果与把所有成员写在一个文件中完全一样。

5.2 应用场景

- 自动生成代码：Unity 中每个场景的脚本自动生成、WinForms/WPF 的 UI 代码都是 `partial class`，开发者只需在另一个文件中手写业务逻辑
- 大型团队合作：不同开发者可以同时编辑同一个类的不同文件，减少合并冲突
- 组织大型类：当一个类太大时，可以按功能拆分到多个文件中，提高可读性

5.3 潜在缺点

- 如果一个类的功能被拆分到太多文件中，理解完整行为变得困难
- 滥用 `partial class` 可能导致职责不清，违反单一职责原则
- 调试时代码跳转可能不如单文件方便

6 Lambda 表达式

6.1 什么是 Lambda 表达式？

[p.23]

Lambda 表达式是一种快速定义匿名函数的方法。可以将其想象成一个”简易函数”，没有函数名（因此也叫匿名函数），可以直接赋值给委托类型变量。

第一种：Expression Lambda

(参数列表) => (返回的表达式)

`(x, y) => x * y` // 等价于: `T MyFunc(T x, T y) { return x * y; }`

使用:

```
Func<int, int, int> mul = (x, y) => x * y;
int result = mul(3, 4); // result = 12
```

第二种：Statement Lambda [p.24]

```
(参数列表) => {
    // 执行的程序语句
}
```

```
// 示例
Action<string> greet = name => {
    string greeting = $"Hello {name}!";
    Console.WriteLine(greeting);
};
greet("World"); // 输出: Hello World!
```

两种 Lambda 的区别:

- Expression Lambda: 只能包含一个表达式，该表达式的结果自动作为返回值。适合简短操作。
- Statement Lambda: 可以包含多条语句（放在花括号中）。如果需要返回值，使用 `return` 语句。如果没有返回值，可以赋值给 `Action` 类型。

6.2 函数作为参数

[p.25]

Lambda 表达式的一个强大之处在于可以轻松地将函数作为参数传入另一个函数:

```
// 定义一个延迟执行的协程
IEnumerator DelayedCall(Action action, float seconds) {
    yield return new WaitForSeconds(seconds);
    action(); // seconds秒后执行传入的action
}

// 调用：2秒后销毁GameObject
StartCoroutine(DelayedCall(() => {
    Destroy(gameObject);
}, 2.0f));
```

逐行分析上面的调用代码：

1. `() => { Destroy(gameObject); }` ——创建一个无参数的 Lambda，执行销毁操作
2. 这个 Lambda 的类型是 `Action`（无参数、无返回值）
3. 它被作为第一个参数传给 `DelayedCall`
4. 2 秒后，协程调用 `action()`，即执行 Lambda 体中的代码

这种模式在 Unity 中的应用：

- 延迟执行（`DelayedCall` 模式）
- UI 按钮回调：`button.onClick.AddListener(() => { ... });`
- 动画完成回调
- 网络请求回调

7 隐式类型（Implicit Type）

7.1 var 关键字

[p.26]

`var` 关键字允许我们在声明变量时**不显式写出类型**，让编译器从赋值表达式中推断出来：

```
var a = 5;           // 编译器推断为 int
var b = 3.14f;       // 编译器推断为 float
var c = "Hello!";    // 编译器推断为 string
var d = gameObject;  // 编译器推断为 GameObject
```

重要：`var` 不是在声明“动态类型”——变量仍然有明确的静态类型，只是由编译器自动推断。编译后和显式声明类型完全一样。

7.2 foreach 循环

[p.26]

foreach 是一种简洁遍历集合的方式。与 var 搭配时极为方便：

```
// foreach + var
foreach (var item in list) {
    item += 5;
}

// 等价的for循环
for (int i = 0; i < list.Count; i++) {
    list[i] += 5;
}
```

foreach vs for:

- foreach 更简洁、更安全（不会出现索引越界）
- foreach 遍历时不能修改集合结构（增加/删除元素会抛出异常）
- for 更灵活（可以控制遍历方向、步长等）

8 课后习题详解

[p.27]

8.1 题 1: 编写一个函数，在 A 秒后执行函数 F，之后每 B 秒再执行一次 F

```
IEnumerator RepeatCall(Action action, float delayA, float intervalB) {
    yield return new WaitForSeconds(delayA); // 等待A秒
    while (true) {
        action(); // 执行函数F
        yield return new WaitForSeconds(intervalB); // 每B秒重复
    }
}

// 使用
StartCoroutine(RepeatCall(() => {
    Debug.Log("执行！");
}, 2.0f, 1.0f));
// 2秒后首次执行，之后每1秒执行一次
```

设计要点：

1. 使用 `Action` 作为参数类型——接受无参数无返回值的函数（和 `lambda`）
2. `delayA = 2f` 控制首次延迟，`intervalB = 1f` 控制重复间隔
3. `while(true)` 实现无限循环——调用方可以通过 `StopCoroutine()` 来停止
4. 如果需要在支持停止条件，可以改为接受 `Func<bool>` 或添加计数器

8.2 题 2：什么时候使用属性，什么时候使用字段？**优先使用属性（Property）的场景：**

- 需要公开暴露的数据——属性提供更好的封装性
- 将来可能需要在读写时添加验证、计算、日志
- 需要在调试时设断点观察数据变化
- 数据绑定（WPF/Unity UI）通常需要属性支持

优先使用字段（Field）的场景：

- 类的私有内部状态——外部不需要访问
- 需要作为 `ref / out` 参数传递
- 高性能场景——字段访问比属性调用稍快（虽然 JIT 通常会内联简单的 `getter/setter`，差异极小）

总结：对外公开的数据接口用属性，私有的内部实现用字段。这遵循了“封装变化”的面向对象原则——将来如果需要添加读写逻辑，只需修改属性定义而不影响所有调用者。

8.3 题 3：删除 List 中所有小于 0 的数**错误写法（常见坑）：**

```
// 错误！foreach遍历时修改集合会抛出InvalidOperationException
foreach (var item in list) {
    if (item < 0) list.Remove(item);
}
```

正确写法 1 ——倒序 for 循环：

```
for (int i = list.Count - 1; i >= 0; i--) {
    if (list[i] < 0) {
        list.RemoveAt(i);
    }
}
```

为什么倒序？ 如果正序删除，索引 *i* 处元素被移除后，后面的元素会向前移动，导致某些元素被跳过。

正确写法 2 ——RemoveAll 方法（推荐）：

```
list.RemoveAll(x => x < 0);
```

`RemoveAll` 是 `List<T>` 提供的方法，接受一个 `Predicate<T>` 委托（等同于 `Func<T, bool>`），在内部使用高效的算法一次遍历完成删除。它是最简洁、最高效的写法。

写法 2 的 Lambda 分析： `x => x < 0` 是一个 Expression Lambda，接受 `int x`，返回 `bool`（`x` 是否小于 0）。其类型是 `Predicate<int>`（等价于 `Func<int, bool>`）。